12. gyakorlat

Python kurzus

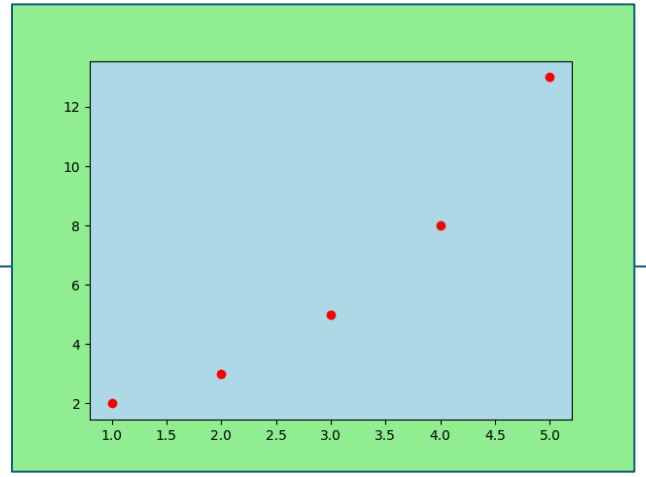
Neurális hálók

Deep Learning

Néhány további adatvizualizációs beállítás:

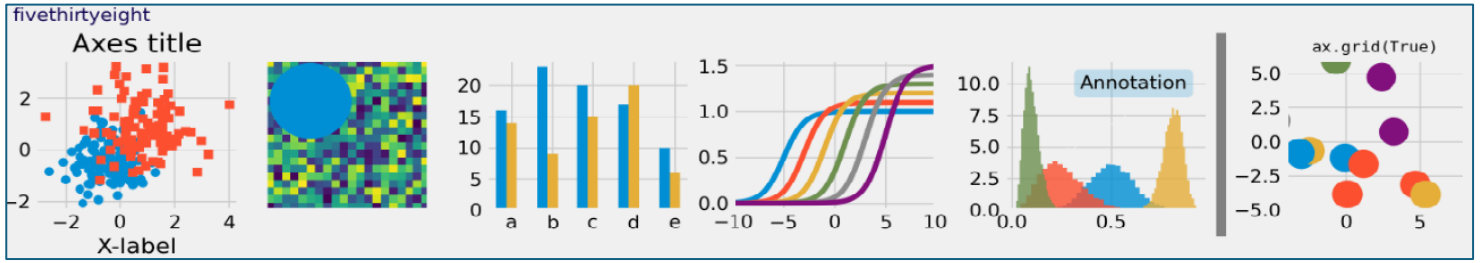
1. Háttérszínek beállítása

```
import matplotlib.pyplot as plt
import pandas as pd
# Példa DataFrame létrehozása
data = {'X': [1, 2, 3, 4, 5], 'Y': [2, 3, 5, 8, 13]}
df = pd.DataFrame(data)

# Diagram létrehozása
fig = plt.figure(facecolor='lightgreen')  # A teljes ábra háttérszínének beállítása
ax = plt.gca()            # Az aktuális tengelyek lekérése: get current axis
ax.set_facecolor('lightblue')              # A rajzterület háttérszínének beállítása

# Adatok ábrázolása
plt.plot(df['X'], df['Y'], 'ro')

# Diagram megjelenítése
plt.show()
```



2. Stílusok beállítása

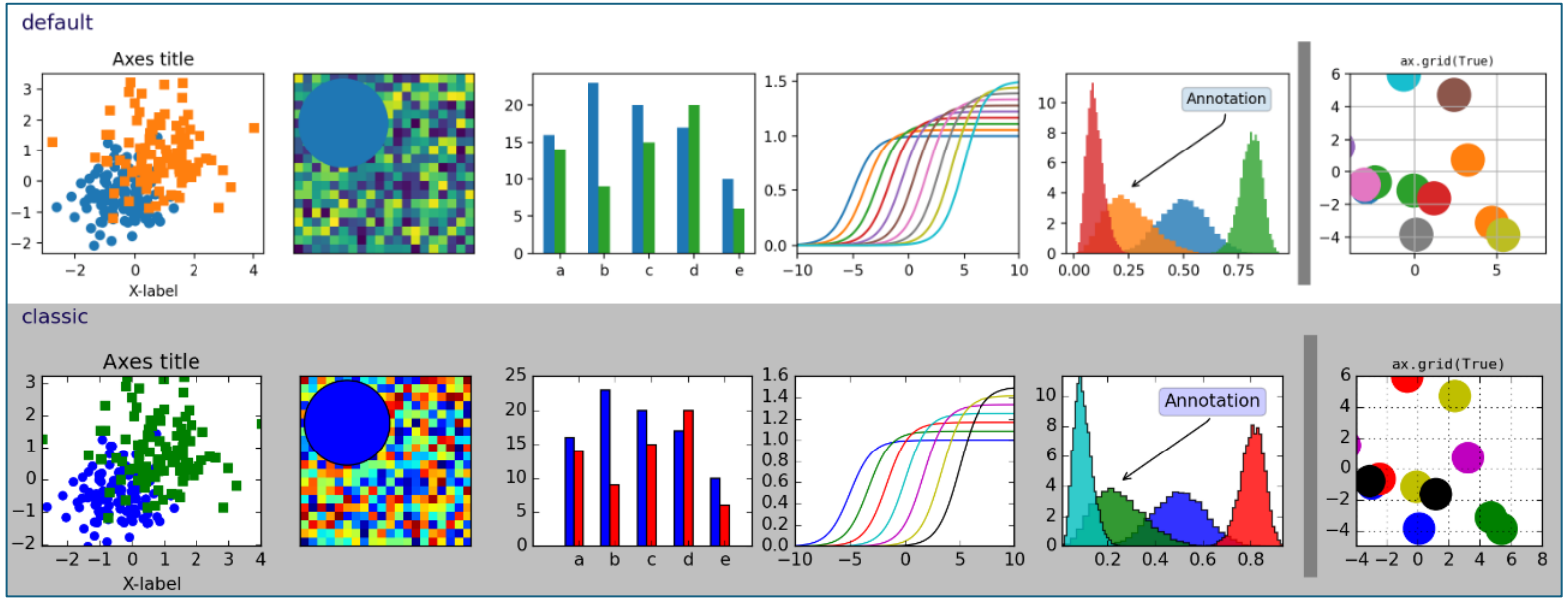
```
# Stílus beállítása
plt.style.use('fivethirtyeight')
```



A **plt.style.use()** paraméterével stíluslapot választhatunk a megjelenítéshez az alapbeállítástól eltérően.

[Style sheets reference — Matplotlib 3.9.2 documentation]

Példák:



ML algoritmusok alapvető fogalmai 1.

- **Felügyelt tanulás** (Supervised): alkalmazunk célváltozót a modellben, ahol X a bemeneti adataink/jellemzők (features), és y a hozzájuk tartozó címkék/értékek (target). Az X-nek 2D tömböt kell adni, és a kimenet az osztályok/értékek 1D tömbje lesz. A cél egy olyan függvény megtanulása, amely a bemenetből a helyes kimenetet meg tudja becsülni.

Példák:
  - Osztályozási algoritmusok (**Classification**): LogisticRegression, DecisionTree, RandomForest, SVM.
  - Regressziós algoritmusok (**Regression**): LinearRegression, SVR, XGBoost.

- **Nem felügyelt tanulás** (Unsupervised): nincs célváltozó a modellben. Csak bemeneti adatok vannak, címkék nincsenek. A cél az adatok belső szerkezetének, mintázatainak felfedezése (klaszterezés, dimenziócsökkentés, anomália-detekció).

Példák:
  - Klaszterezés algoritmusok (**Clustering**): KMeans, DBSCAN, Agglomerative.
  - Dimenziócsökkentés algoritmusok: PCA, TSNE, UMAP.
  - Anomália detekció: IsolationForest, OneClassSVM.

ML algoritmusok alapvető fogalmai 2.

- **Skálázás / Normalizálás:** Jellemzők egységesítése, fontos sok algoritmusnál. (StandardScaler, MinMaxScaler, …)

- **Modellépítés:** tanító és teszt (train-test) adathalmazok szétválasztása, az algoritmus betanítása, eredmények megjelenítése.

- **A modell teljesítményének értékelése:** metriák és validációs eszközök alapján:
  - **Pontosság (accuracy):** A helyes előrejelzések aránya az összes előrejelzéshez képest.
  - **Precízió (precision):** A helyes pozitív előrejelzések aránya az összes pozitív előrejelzéshez képest.
  - **Recall:** A helyes pozitív előrejelzések aránya az összes valós pozitív esethez képest.
  - **F1-score:** Precízió és Recall harmonikus átlaga.

  Értékelő metrikák:
  - Osztályozásnál: accuracy, precision, recall, F1, ROC-AUC,
  - Regressziónál: MSE, MAE, R^2.

- **Konfúziós mátrix** (Confusion Matrix): Hibaelemzéshez, az osztályozási kimenetek részletes hibamegoszlása (TP, FP, TN, FN).

Konfúziós mátrix (Confusion matrix) felépítése:

A konfúziós mátrix egy négyzet alakú mátrix, amelynek oszlopai a valós osztályokat, sorai pedig a modell által előrejelzett osztályokat reprezentálják. Az egyes cellák számai azt mutatják, hogy hány adatpont tartozik az adott valós és előrejelzett osztály kombinációhoz.

Kétosztályos (bináris) osztályozás esetén:

	Valódi Pozitív	Valódi Negatív
Pozitív predikció	TP	FP
Negatív predikció	FN	TN

- TP (True Positive): Helyesen pozitívnak előrejelzett esetek száma.
- TN (True Negative): Helyesen negatívnak előrejelzett esetek száma.
- FP (False Positive): Tévesen pozitívnak előrejelzett esetek száma (hamis riasztás).
- FN (False Negative): Tévesen negatívnak előrejelzett esetek száma (kihagyott riasztás).

A konfúziós mátrix adataiból a teljesítménymutatók értékei kiszámíthatók a definíciók szerint:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

ML algoritmusok legalapvetőbb sklearn metódusai:

- **train_test_split()** adatok véletlen felosztása train/test halmazra

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=..., random_state=...)
```

- **fit(X_train, y_train)** a modell betanítása a train adatokon

```
model = model.fit(X_train, y_train)
```

- **predict(X_test)** előrejelzések készítése a teszt adatokra

```
y_pred = model.predict(X_test)
```

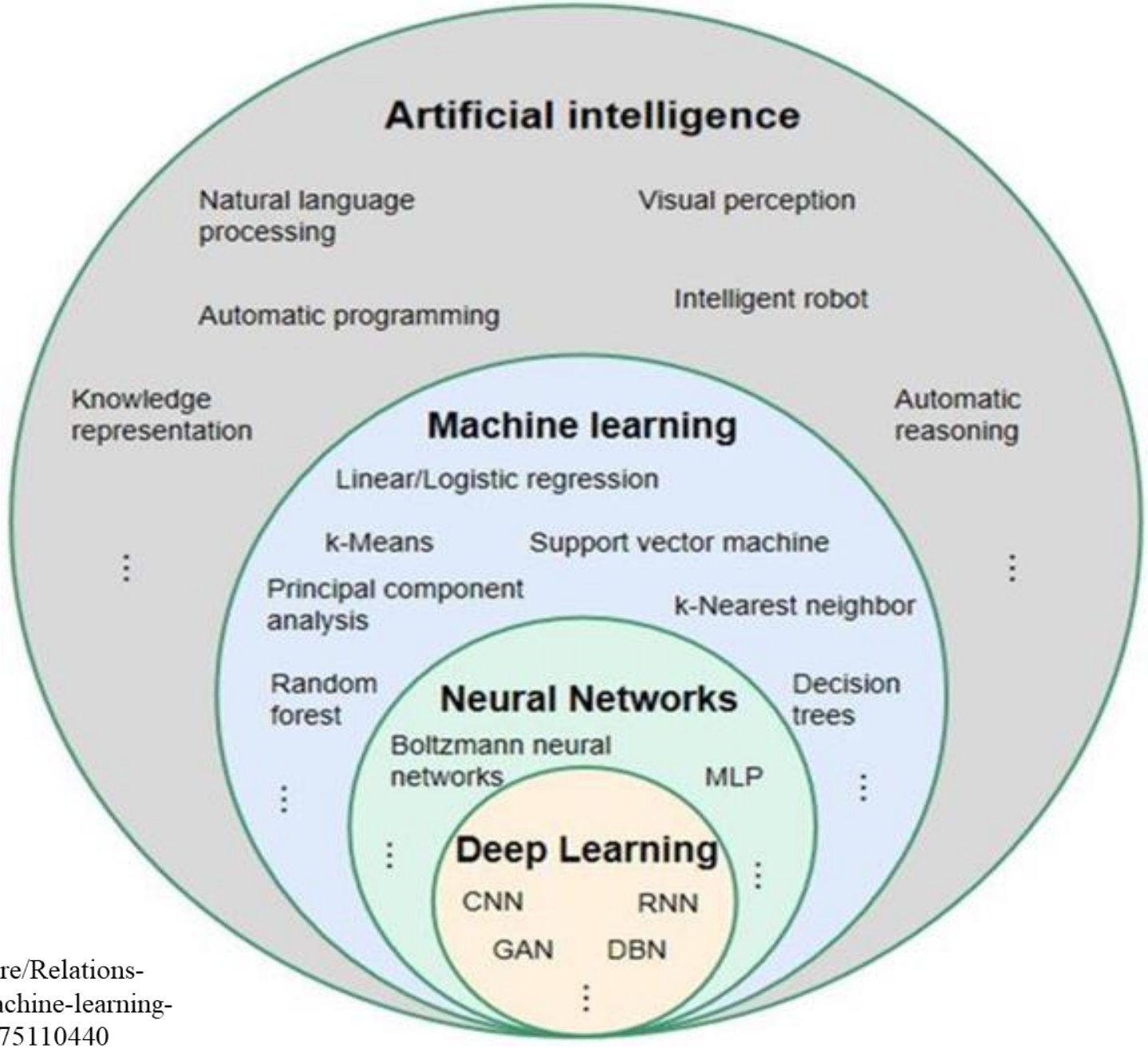
- **score(X_test, y_test)** regressziónál megadja a modell pontosságát

```
pontossag = model.score(X_test, y_test)
```

- **accuracy_score(y_test, predikciok)** klasszifikáció pontosságának megadása

```
predikciok = model.predict(x_teszt)
pontossag = accuracy_score(y_test, predikciok)
```

(valós tesztcímkék, predikciók)



https://www.researchgate.net/figure/Relations-between-artificial-intelligence-machine-learning-neural-network-and-deep_fig2_375110440

Mélyhálók és a Deep Learning alapfogalmai

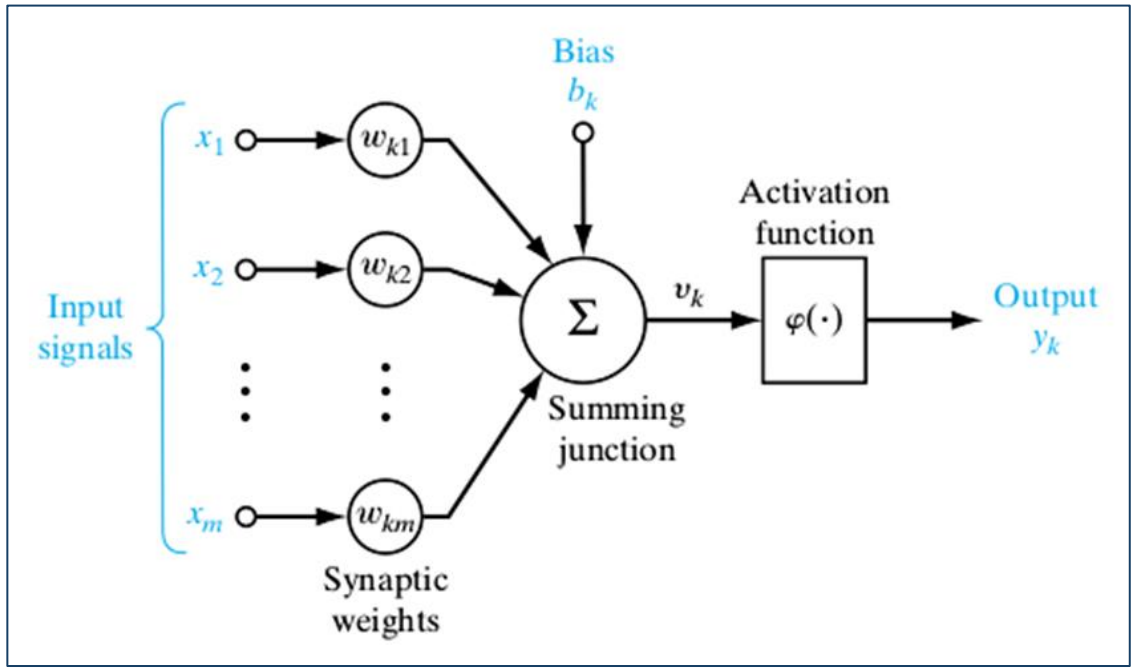
1. Neurális hálózatok

Többrétegű modell: rétegek (layer) és mesterséges neuronok (unit) hálózata, a Deep Learning alapépítménye.

- **Rétegek:**
  - **Bemeneti réteg:** Az adatokat fogadja (pl. egy kép pixeleinek intenzitása)
  - **Rejtett rétegek:** Feldolgozzák az adatokat, felismerve a **mintázatokat.**
  - **Kimeneti réteg:** Az eredményeket adja vissza, pl. egy osztályozás eredményét (pl. számjegyfelismerés esetén a 0–9 számjegyek).
- **Neuronok:** A neurális hálózat elemei, amelyek **egy-egy értéket számolnak ki.** A neuronok a **bemeneteket súlyozzák**, majd egy **aktivációs függvényen** keresztül eredményt adnak.
- **Kapcsolatok:** A neurális hálózatban a neuronokat **élek (kapcsolatok)** kötik össze. Minden kapcsolat egy **súlyértékkel** rendelkezik, amely a tanulási folyamat során **frissül**.
- **A mélytanulási modellek** több rejtett réteggel rendelkeznek, amelyeken keresztül az adatokat feldolgozzák.

A mesterséges neuronok szerkezete:

- Bemenetet fogad a szinapszison (x_i) keresztül (a dendriteken):
  - a **bemenetek súlyozottak** (w_i)
  - ha $w_i > 0$: erősített bemenet az adott forrásból (gerjesztő bemenet)
  - ha $w_i < 0$: csillapított bemenet az adott forrásból (gátló bemenet)
- **Súlyozott összeget** számítanak ki
- A **b érték torzítja** az összeget, hogy lehetővé tegye az aszimmetrikus viselkedést
- **Aktivációs függvény** formálja a kimenetet



x_i: bemeneti vektor
w_{ki}: a k neuron súlyegyüttható-vektora
b_k: a k neuron torzítási értéke
y_k: a k neuron kimeneti értéke

Kimeneti egyenlet:

$$y_k = \varphi\left(\sum_{i=1}^{m} w_{ki}x_i + b_k\right)$$

$w_{k0}=b_k$
$x_0=1$

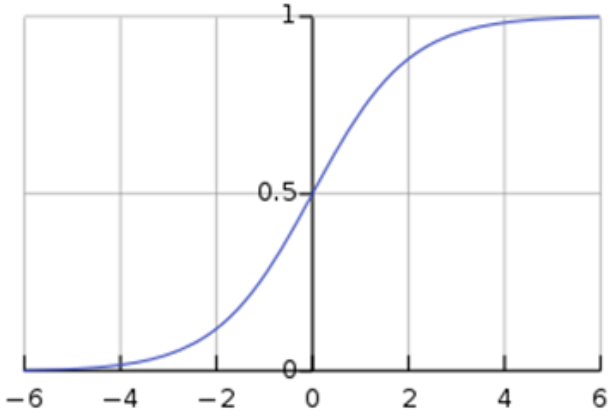
$$y_k = \varphi\left(\sum_{i=0}^{m} w_{ki}x_i\right) = \varphi(\mathbf{w}^T\mathbf{x})$$

2. Aktivációs függvények

Az aktivációs függvények határozzák meg, hogy egy neuron kimenete hogyan alakuljon a következő réteg számára. Az aktivációs függvény nemlinearitást visz a hálózatba, és segít abban, hogy a neurális háló komplex mintázatokat tanuljon meg.

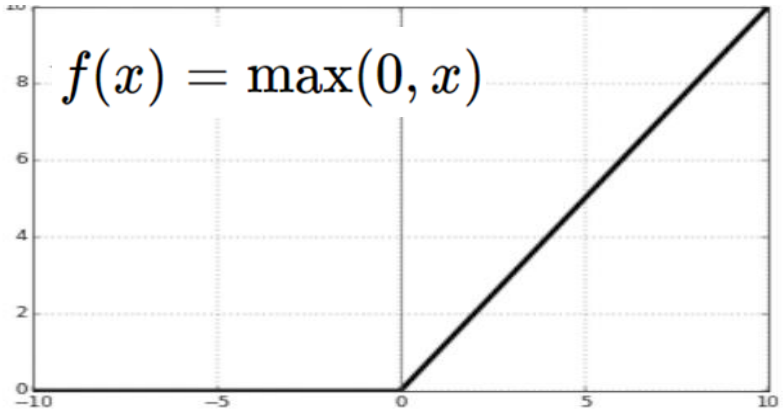
- **Sigmoid:** Az értékeket 0 és 1 közé skálázza.
- **ReLU (Rectified Linear Unit):** Minden negatív bemenetet 0-ra állít, a pozitív értékeket változatlanul hagyja.
- **Softmax:** A kimeneti osztályok valószínűségi eloszlását számítja ki (összegük 1).

Sigmoid

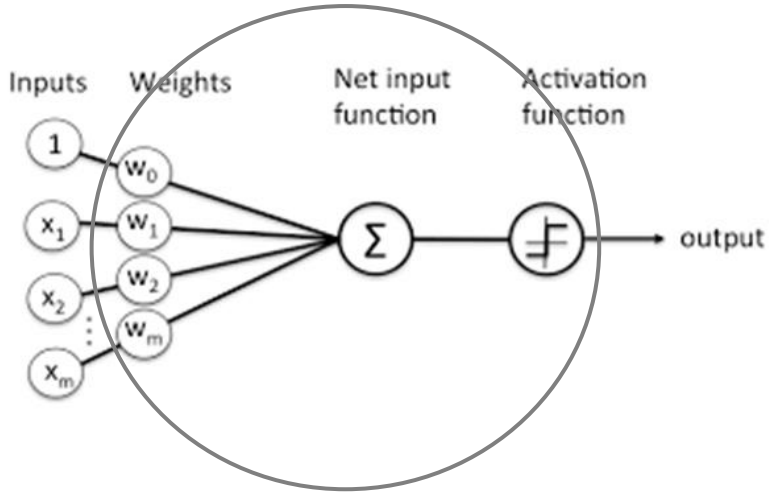


$$S(x) = \frac{1}{1+e^{-x}}$$

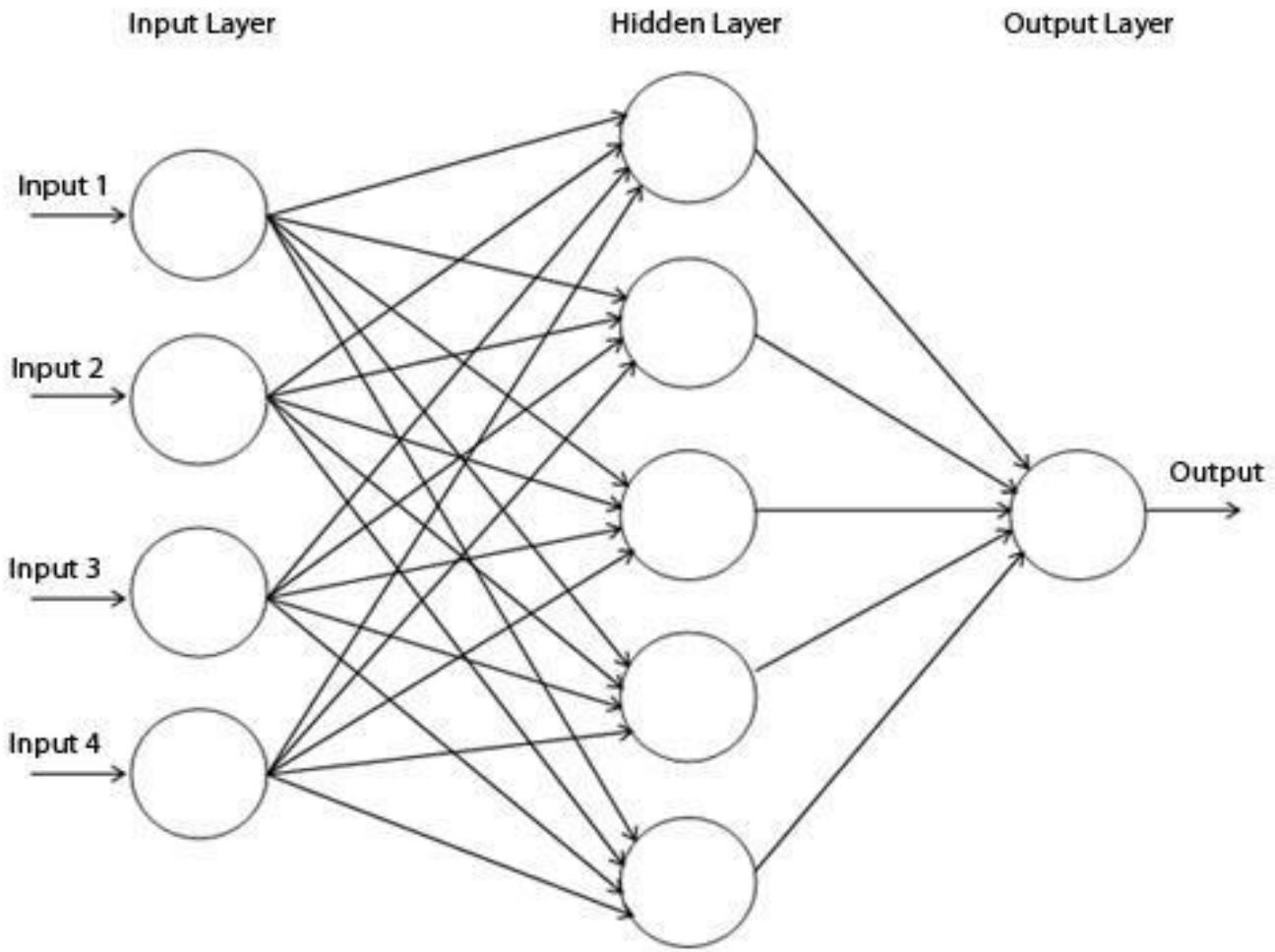
ReLU



Többrétegű mesterséges neurális hálózat (Multi-layer Artificial Neural Network)



mesterséges neuron



4 – 5 – 1 mesterséges neuronból álló rétegek

https://www.inf.u-szeged.hu/~tothl/ann/03.%20Neuronhalok%20es%20tanitasuk.pdf

Visszaterjesztés (Backpropagation) algoritmus – 1986, Geoffrey Hinton és társai

A visszaterjesztés algoritmus célja, hogy **optimalizálja a neurális hálózat súlyait** úgy, hogy minimalizálja a hálózat által elkövetett hibát. Az algoritmus két fő lépésből áll:

1. Előre terjesztés (Forward Propagation):

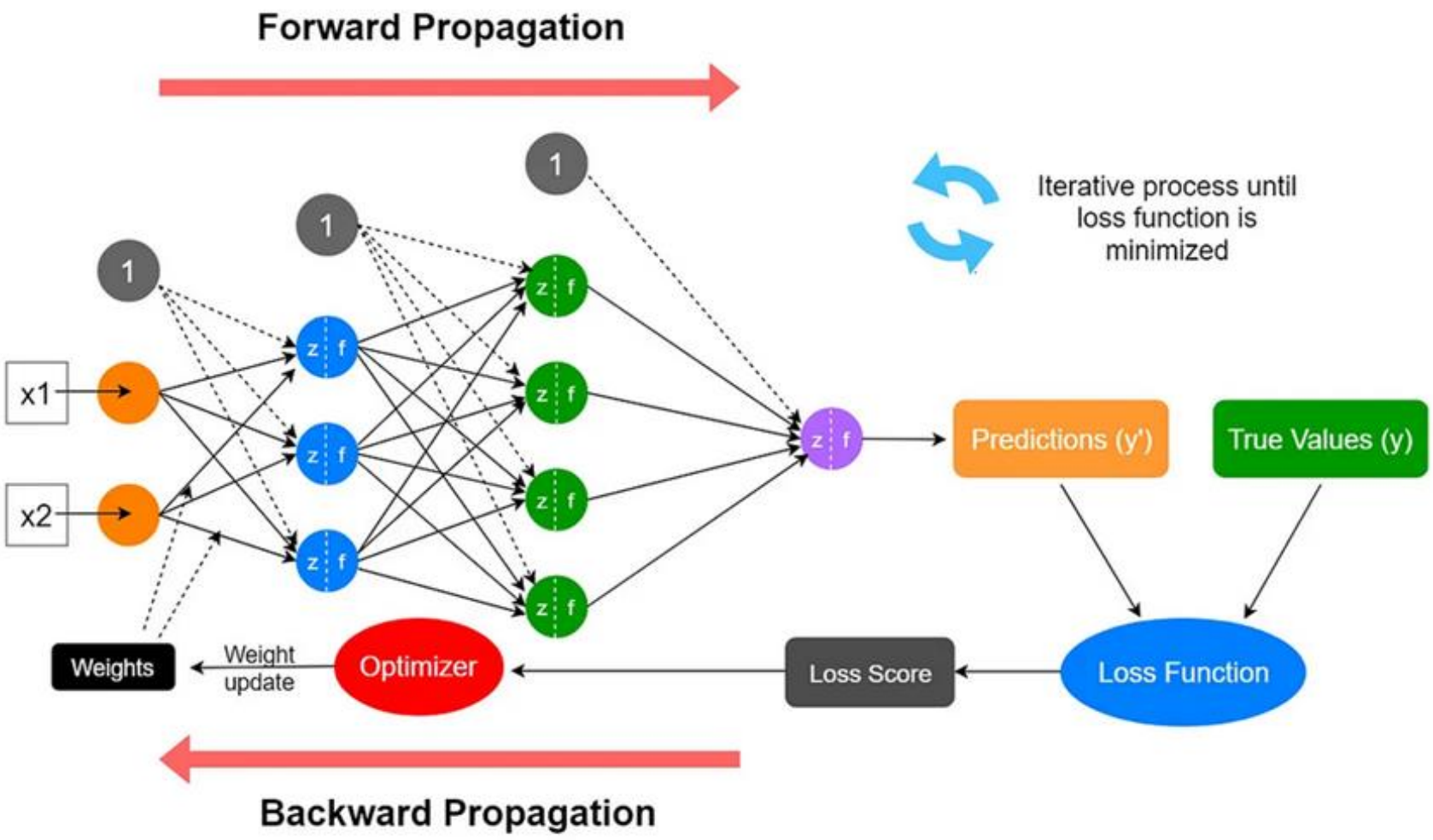
- Az adatok áthaladnak a hálózaton, és minden réteg kiszámítja a kimenetét a bemeneti adatok és a súlyok alapján.
- Az utolsó réteg kimenete a hálózat előrejelzése.

2. Visszaterjesztés (Backpropagation):

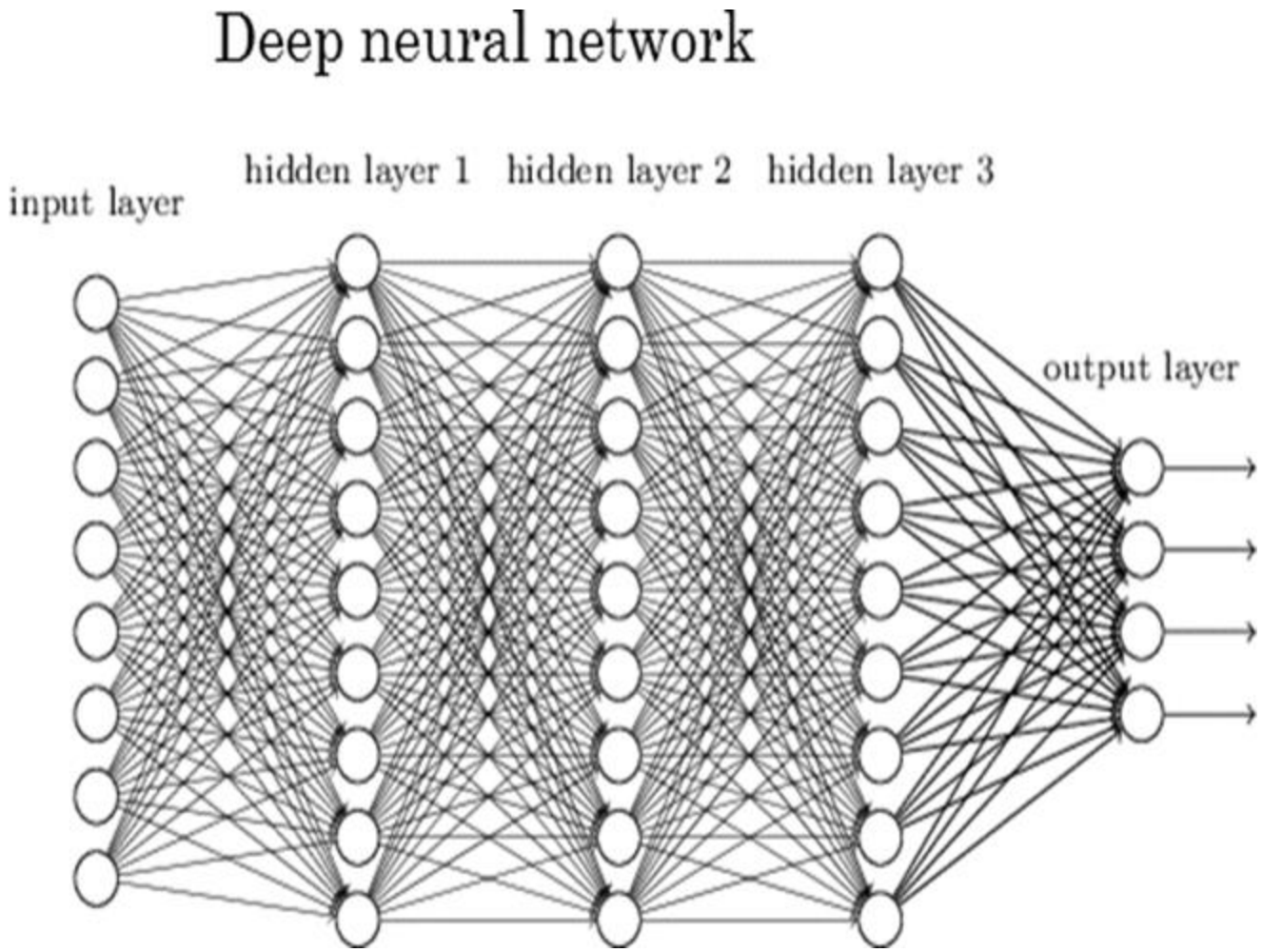
- A visszaterjesztés során a hálózat kiszámítja a hibát (a valós és az előrejelzett érték közötti különbséget) és visszaterjeszti ezt a hibát a hálózaton keresztül.
- A hibát minden rétegben felhasználják a súlyok frissítésére, hogy csökkentsék a hibát. Ez a gradiens csökkenés (**gradient descent**) algoritmus segítségével történik.

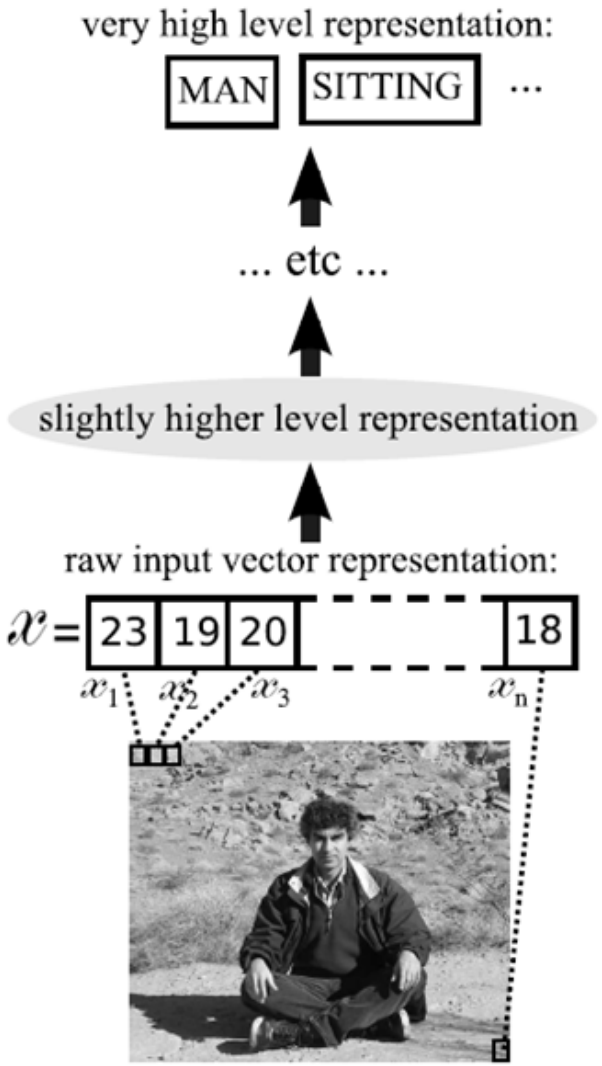
A visszaterjesztés algoritmusa lehetővé teszi a mély neurális hálózatok **hatékony betanítását**, mivel képes kezelni a hálózatban lévő sok réteg súlyainak optimalizálását. Lásd a következő dia ábráját.

Az ANN (Artificial Neural Network) tanulási folyamata



https://www.researchgate.net/figure/The-Forward-Propagation-Process-of-Neural-Network_fig4_382298512





https://www.researchgate.net/figure/The-structure-of-a-deep-neural-network_fig2_382298512

https://www.researchgate.net/figure/Picture-illustrating-the-distance-between-the-raw-data-and-high-level-description-of_fig2_306057143

A TensorFlow könyvtár alapjai

TensorFlow definíciója:

A TensorFlow egy nyílt forráskódú, skálázható és hatékony **gépi mélytanulási keretrendszer**, amelyet a Google hozott létre. Különböző platformokon működik, és támogatja a mély neurális hálózatok fejlesztését és futtatását.

Jellemzői:

1. Skálázhatóság: Kis mintáktól kezdve a nagy adatbázisokig mindenre alkalmas.
2. Hatékonyság: Optimalizált algoritmusokat használ a gyors tanításra és futtatásra.
3. Támogatott platformok: Mobiltelefonokon, felhőben és böngészőben is használható.
4. Rugalmasság: Támogatja az egyszerű magas szintű API-kat és az alacsony szintű programozást. **Keras API**-t használ a mélyhálók alkalmazásához.

TensorFlow hivatalos dokumentáció: A TensorFlow API részletes magyarázata, kódpéldák, és gyakorlatok: https://www.tensorflow.org

Keras dokumentáció: Keras API, amely megkönnyíti a TensorFlow-val való fejlesztést.

https://keras.io

TensorFlow telepítésének lépései és ellenőrzés

1. Virtuális környezet kialakítása: **python –m venv venv**
2. Virtuális környezet aktiválása: **venv\Scripts\activate**
3. TensorFlow telepítése: **pip install tensorflow**

   (A Keras API a TensorFlow része, nem kell külön telepíteni.)
4. Telepítés ellenőrzése: **import tensorflow as tf**
   print(tf.__version__)

TensorFlow alapfogalmak:

1. Gráf-alapú működés

A TensorFlow **számítási gráfokat** használ, amelyek csomópontokból és élekből állnak. Ez a struktúra meghatározza az adatok áramlását és lehetővé teszi az optimalizációt.

2. Tensorok használata

A TensorFlow központi elemei a tensorok, amelyek n-dimenziós tömböket vagy mátrixokat jelentenek.

A tensorok az adatokat reprezentálják és tárolják, például:

- Egyetlen szám (skalár)
- Vektorok
- Mátrixok és összetettebb struktúrák

Példák tensor létrehozására a **tf.constant()** metódussal:

```
import tensorflow as tf
pelda_tensor = tf.constant([[1, 2, 3], [4, 5, 6]])
print(pelda_tensor)
```

```
tf.Tensor(
[[1 2 3]
 [4 5 6]], shape=(2, 3), dtype=int32)
```

```
a = tf.constant([3, 5])
b = tf.constant([4,2])
szorzat = a * b
print(szorzat)
```

```
tf.Tensor([12 10], shape=(2,), dtype=int32)
```

Példa: Egyszerű tensor művelet

Az a és b tensorok létrehozása és az **add()** matematikai művelet végrehajtása

```
import tensorflow as tf
# Tensorok létrehozása
tensor_a = tf.constant([5, 10, 15], name="tensor_a")
tensor_b = tf.constant([2, 4, 6], name="tensor_b")
# Alapvető matematikai művelet
osszeg = tf.add(tensor_a, tensor_b)
# Eredmény kiíratása
print("Eredmény:", osszeg.numpy())
```

Eredmény: [7 14 21]

A **numpy()** metódus átalakítja a tensort NumPy tömbbé.

Neurális hálózatok felépítése TensorFlow-val

Az alapstruktúra rétegei: A neurális hálózatokat **három fő rétegtípus** alkotja, amelyek együttműködnek a bemenetek feldolgozásában és a kimenetek előállításában:

1. **Input réteg:** A bemeneti réteg fogadja az adatokat. Ez határozza meg a modell bemeneti dimenzióit.
   Példa: Egy képfeldolgozó modell bemenete 28x28 pixel lehet, ahol minden pixel intenzitását egy érték képviseli.
2. **Rejtett rétegek: F**eldolgozzák a bemenetet a tanulási folyamat során. Több rejtett réteg is lehet. Feladatuk az adatban található mintázatok felismerése.
3. **Kimeneti réteg:** Az eredmény előállítása.
   Példa: Ha a modell célja egy képen lévő tárgy osztályozása, a kimeneti réteg a kategóriák valószínűségeit adja vissza.

Aktivációs függvények (φ)

Az aktivációs függvények adják meg, hogy az egyes rétegek hogyan alakítsák át a bemenetet a következő réteg számára (**ReLU, Sigmoid, Softmax**)

Neurális hálózat felépítése TensorFlow-ban:

A TensorFlow-ban a **Sequential API**-val a rétegek egymás után adhatók hozzá a modellhez.

1. Sequential modell: egy egyszerű, egymás után fűzött rétegekből álló „lineáris” modell

2. Rejtett rétegek:

- **Dense** (Sűrű) **réteg:** Minden bemenet kapcsolódik az összes kimenethez.
- **Dropout réteg:** Meghatározott arányú neuront véletlenszerűen kikapcsol a túltanulás elkerülése érdekében.

3. Rétegek hozzáadása sorban: modell.add(Dense(128, ...))

4. A modell összegzése: modell.summary() megmutatja a rétegeket és paramétereket.

5. A modell futtatása:

- inicializáljuk a modellt, és sorban hozzáadjuk a rétegeket a bemeneti réteghez;
- fordítás **compile(...)**-lal, paraméterek: optimalizáló, a veszteségfüggvény, metrikák;
- betanítás **fit(...)**-tel;
- értékelés **evaluate(...)**-tel;
- predikció **predict(...)**-tel.

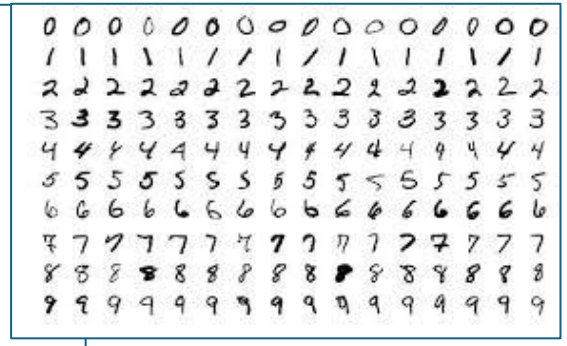
Neurális hálózat felépítése: egyenes illesztés, a y = 2x - 1 egyenlet tanulása

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense
```

```
layer0 = Dense(units=1, input_shape=[1]) #egy paraméter egy inputtal

model = Sequential([layer0]) #szekvenciális modell a rétegekkel
model.compile(optimizer='sgd', loss='mean_squared_error')
# optimizálós algoritmus, veszteségfüggvény, összerakjuk a modell alapját(compile)

# Értékeket adunk meg a tanításhoz, megfelelő címkeszámmal
x_input = np.array([-1.0, 0.0, 1.0, 2.0, 3.0, 4.0], dtype=float) # x
y_target = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0], dtype=float) # 2x-1

model.fit(x_input, y_target, epochs=500, verbose=0) #itt tanítjuk a modellt

#adott réteg súlyaival és eltolásával (bias) megnézzük a tanult paramétereket
print(f"Tanult súlyok: {layer0.get_weights()}")  # kb 2 (szorzó), kb -1 (bias)

print(model.predict(np.array([10.0]))) # 19-nek kellene kijönnie
```

1. példa: Egyszerű neurális háló felépítése képosztályozásra (az MNIST adatkészlettel)

```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Dropout

modell = Sequential()          # Modell inicializálása

# Az első rejtett réteg hozzáadása a bemeneti réteghez
modell.add(Dense(128, activation='relu', input_shape=(784,), name="rejtett_reteg_1"))

# Dropout a túltanulás elkerülésére
modell.add(Dropout(0.2, name="dropout_reteg"))

# Második rejtett réteg
modell.add(Dense(64, activation='relu', name="rejtett_reteg_2"))

# Kimeneti réteg (10 osztály esetén)
modell.add(Dense(10, activation='softmax', name="kimeneti_reteg"))
# A modell struktúrájának megjelenítése
modell.summary()
```



https://en.wikipedia.org/wiki/File:MnistExamples.png

Magyarázat:

- **Dense:** A rejtett rétegek neuronjainak számát 128 és 64 értékre állítottuk be. Az aktivációs függvény a **ReLU**. Az **input_shape** az első rétegnél a bemenet dimenzióját határozza meg (28x28=784 pixelből álló képek).

- **Dropout:** A réteg kikapcsolja a neuronok 20%-át minden tanítási iterációban.
- A kimeneti réteg az osztályozásért felelős, a **softmax** aktivációs függvény pedig az osztályok valószínűségét adja meg.

- **A modell struktúrája - modell.summary():**

Model: "sequential"

Layer (type)	Output Shape	Param #
rejtett_reteg_1 (Dense)	(None, 128)	100,480
dropout_reteg (Dropout)	(None, 128)	0
rejtett_reteg_2 (Dense)	(None, 64)	8,256
kimeneti_reteg (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

A modell fordítása: a paraméterek inicializálása és konfigurálása:

```
# Modell fordítása
modell.compile(
   optimizer='adam',                    # Optimalizációs algoritmus
   loss='categorical_crossentropy’,     # Veszteségfüggvény többosztályos osztályozáshoz
   metrics=['accuracy']                 # Teljesítménymetrika
)
print("Modell inicializálva és lefordítva.")
```

- **Optimizációs algoritmus:** Frissíti a modell paramétereit (a súlyokat) a gradiens-alapú optimalizáció segítségével. Az **Adam** (Adaptive Moment Estimation) optimizáló hatékony és gyorsan konvergál.
- **Veszteségfüggvény:** Az eltérést méri a modell által generált kimenet és a célérték között. A többosztályos osztályozáshoz és a softmax aktivációs függvényhez **categorical_crossentropy**–t (kereszt entrópiát) használjuk.
- **Metrikák:** Az **accuracy** mutatja meg, hogy a modell milyen pontossággal végzi az osztályozást.

2. példa: Egy teljes tanítási és validációs folyamat (az MNIST adatkészlettel):

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Adatelőkészítés, magyarázat a 27. dián
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
x_train = x_train.reshape((-1, 28 * 28)).astype('float32') / 255
x_test = x_test.reshape((-1, 28 * 28)).astype('float32') / 255
y_train = tf.keras.utils.to_categorical(y_train, 10)      # *
y_test = tf.keras.utils.to_categorical(y_test, 10)        # *

# Modell létrehozása (lehet listában megadni a rétegeket)
modell = Sequential([
   Dense(128, activation='relu', input_shape=(784,)),
   Dense(64, activation='relu'),
   Dense(10, activation='softmax')
])
# Modell fordítása
modell.compile(
   optimizer='adam',
   loss='categorical_crossentropy',
   metrics=['accuracy'] )
```

astype('float32'):
Az adatokat lebegőpontos számokká alakítja, hogy kompatibilisek legyenek a TensorFlow modellekkel.

/ 255:
Az adatokat normalizálja 0 és 1 közé, mivel az eredeti képek pixelértékei 0 és 255 között vannak. Így a modell gyorsabban tanul.

***:** 0-9 számok vektorrá kódolása, pl. a 3 kódja: [0,0,0,1,0,0,0,0,0,0].

```
# Modell tanítása, megjelenik a 10 tanítási ciklusnál sorban a batch-ek
száma, a progress bar, az eltelt idő, és a History object (egyes epochok
metrika/ veszteség értékei: accuracy, loss, val_accuracy, val_loss)

eredmeny = modell.fit(
   x_train, y_train,
   epochs=10,
   batch_size=32,
   validation_split=0.2 )
```

```
Epoch 1/10
1500/1500 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8640 - loss: 0.4836 - val_accuracy: 0.9571 - val_loss: 0.1461
```

Az adatelőkészítés fázisainak magyarázata:

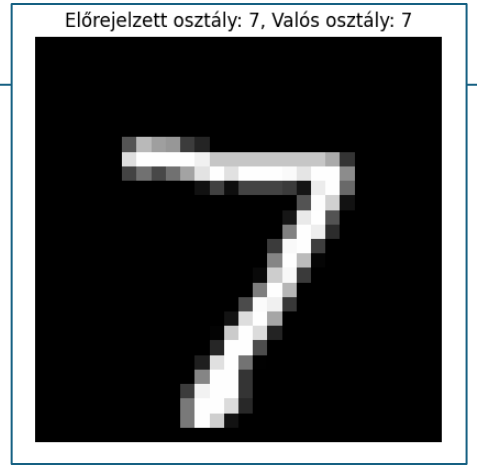
- **tf.keras.datasets.mnist.load_data()** adatok betöltése: NumPy tömböket ad vissza.
- Az adathalmaz **train / test részre** van osztva (60000 / 10000 adat):
- **x_train és x_test:** NumPy tömbök, amelyek a képeket tartalmazzák. Méretük:
  x_train: (60000, 28, 28) (60000 kép, mindegyik 28x28 pixel).
  x_test: (10000, 28, 28) (10000 kép, mindegyik 28x28 pixel).
- **y_train és y_test:** NumPy tömbök, amelyek a címkéket tartalmazzák. Méretük:
  y_train: (60000,) (60000 címke).
  y_test: (10000,) (10000 címke).
- A **címkék 0-tól 9-ig** terjedő egész számok, a képeken látható számjegyeket jelölik.
- **reshape((-1, 28 * 28)):** Az eredeti (28, 28) méretű képeket **egyetlen vektorrá** alakítja (28 * 28 = 784) dimenzióval. A -1 azt jelenti, hogy az első dimenzió változatlan marad.

A tanítás fit(…) metódusának paraméterei:

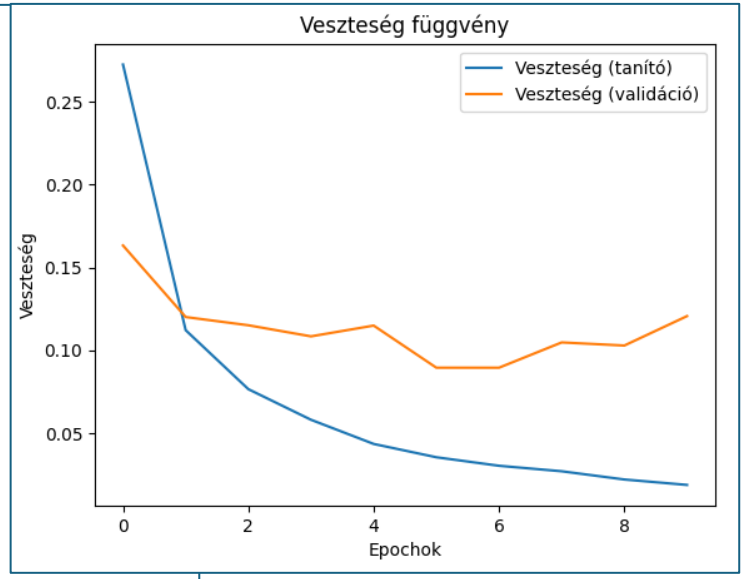
- **epochs=10 →** 10 tanítási ciklus történik az x_train, y_train adatokon
- **batch_size=32 →** 1500 db 32 adatból álló köteg = 48000 adat (a train adatok 80%-a)
- **validation_split=0.2 →** a validációs adathalmaz 12000 adat a 60000-ből (20%)

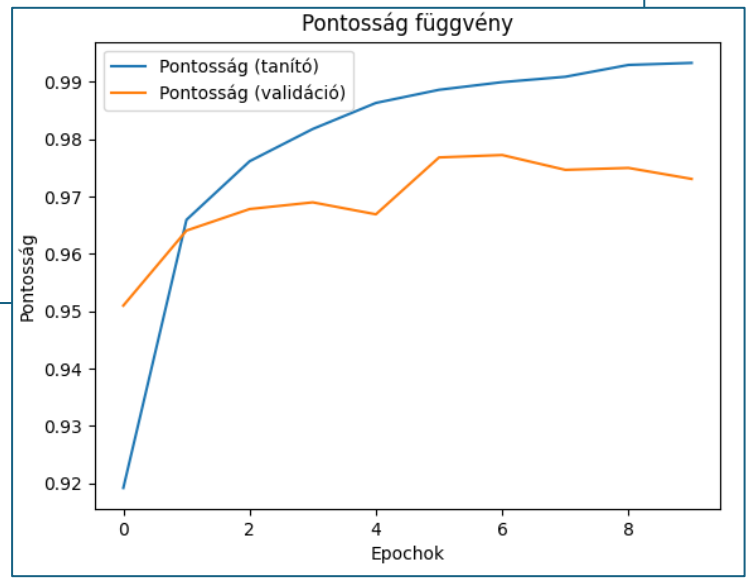
```
# Modell kiértékelése
teszt_veszteseg, teszt_pontossag = modell.evaluate(x_test, y_test)
print(f"Teszthalmaz vesztesége: {teszt_veszteseg}")
print(f"Teszthalmaz pontossága: {teszt_pontossag}")

# Előrejelzés új adatokra: index=0 miatt csak az adatbázis 0. elemét vizsgáljuk meg
index = 0  # Példa index
x = x_test[index].reshape(1, 784)       # Az indexelt adat előkészítése
y_pred = modell.predict(x)              # Előrejelzés
print(f"Előrejelzett osztály: {np.argmax(y_pred)}")  # A legmagasabb valószínűségű osztály kiírása
print(f"Valós osztály: {np.argmax(y_test[index])}")     # A valós osztály kiírása

# Eredmények megjelenítése
plt.imshow(x_test[index].reshape(28, 28), cmap='gray')
plt.title(f"Előrejelzett osztály: {np.argmax(y_pred)}, Valós osztály: {np.argmax(y_test[index])}")
plt.axis('off')
plt.show()
```

```
Teszthalmaz vesztesége: 0.10834143310785294
Teszthalmaz pontossága: 0.9746000170707703
1/1 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0s
51ms/step
Előrejelzett osztály: 7
Valós osztály: 7
```



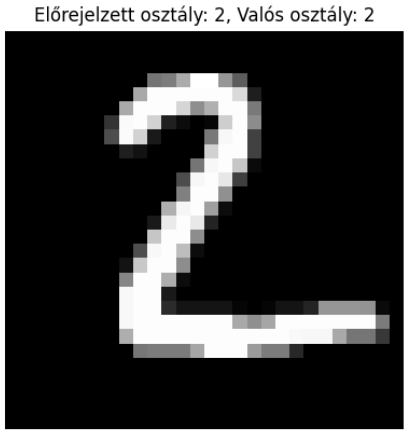
```
# Loss függvény megjelenítése
plt.plot(eredmeny.history['loss'], label='Veszteség (tanító)')
plt.plot(eredmeny.history['val_loss'], label='Veszteség (validáció)')
plt.title('Veszteség függvény')
plt.xlabel('Epochok')
plt.ylabel('Veszteség')
plt.legend()
plt.show()

# Pontosság függvény megjelenítése
plt.plot(eredmeny.history['accuracy'], label='Pontosság (tanító)')
plt.plot(eredmeny.history['val_accuracy'], label='Pontosság (validáció)')
plt.title('Pontosság függvény')
plt.xlabel('Epochok')
plt.ylabel('Pontosság')
plt.legend()
plt.show()
```

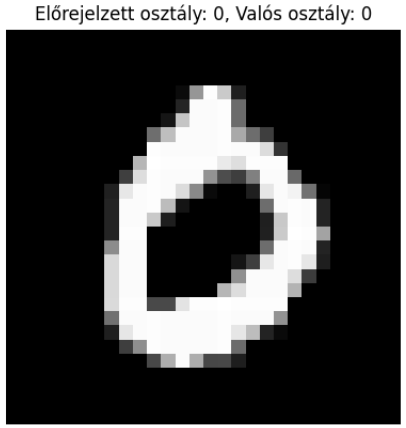


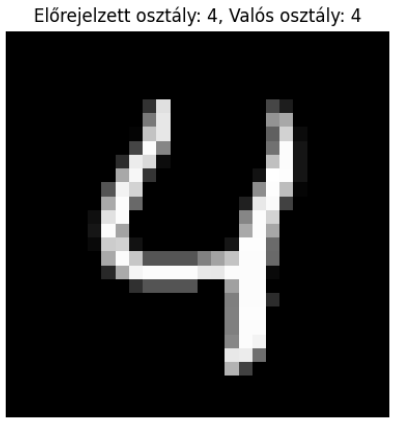


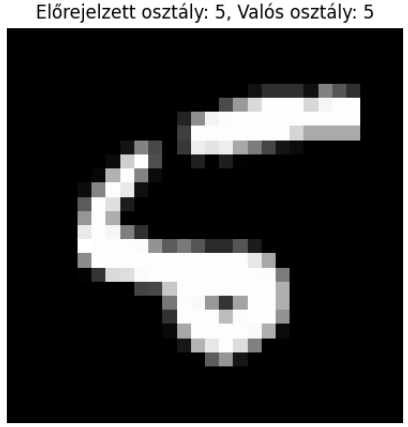
Ciklussal megvizsgáljuk az első 10 tesztadatot:

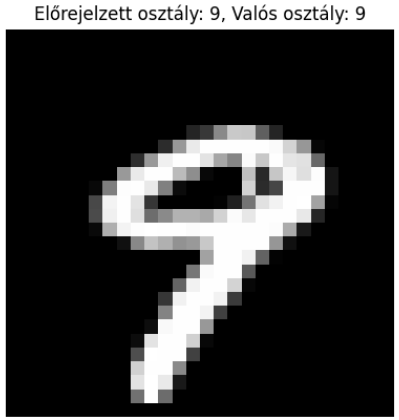
```
for index in range(10):  # Az első 10 tesztadat vizsgálata
    x = x_test[index].reshape(1, 784)  # Az indexelt adat előkészítése
    y_pred = modell.predict(x)  # Előrejelzés
    print(f"Index: {index}")
    print(f"Előrejelzett osztály: {np.argmax(y_pred)}")  # A legmagasabb valószínűségű osztály kiírása
    print(f"Valós osztály: {np.argmax(y_test[index])}")  # A valós osztály kiírása
    print()

    # Az első 10 eredmény megjelenítése
    plt.imshow(x_test[index].reshape(28, 28), cmap='gray')
    plt.title(f"Előrejelzett osztály: {np.argmax(y_pred)}, Valós osztály: {np.argmax(y_test[index])}")
    plt.axis('off')
    plt.show()
```











3. példa: Egyszerű Lineáris Regresszió TensorFlow-ban

```
print('Tensorflow lineáris regresszió’)
# Cél: Egy lineáris függvény (pl. y = 2x + 3) modellezése.

from tensorflow.keras import Sequential, Input
from tensorflow.keras.layers import Dense
# Adatgenerálás (szintetikus)
X = tf.linspace(0.0, 10.0, 100)[:, tf.newaxis]   # 0 és 10 között generál 100 adatpontot
y = 2 * X + 3 + tf.random.normal(X.shape, mean=0.0, stddev=0.5)   # kis zaj hozzáadása a célértékekhez
model = Sequential([
   Input(shape=(1,)),        # Bemeneti forma megadása külön Input rétegként
   Dense(1)                  # 1 Dense réteg
])
```

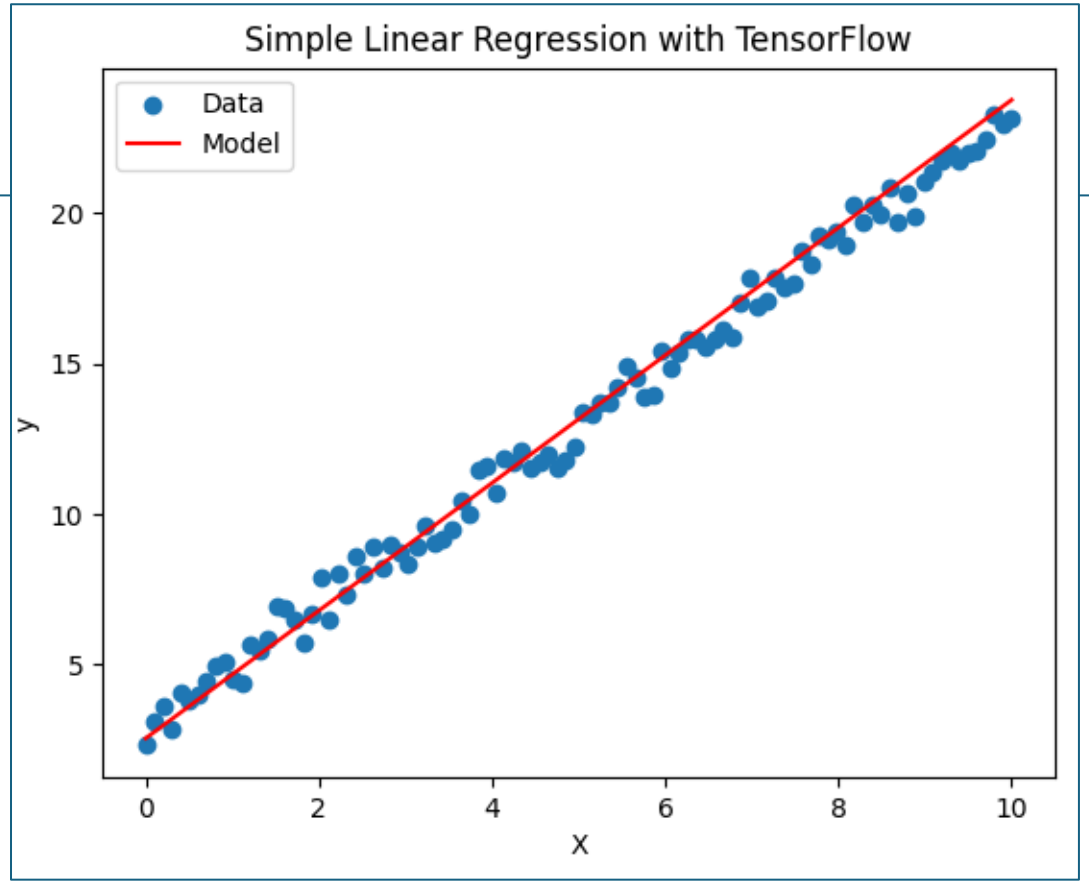
- **X = tf.linspace(0.0, 10.0, 100)[:, tf.newaxis]:** 0 és 10 között generál 100 egyenletesen elosztott adatpontot. Egy új dimenziót ad hozzá a tensorhoz, így az alakja [100, 1] lesz: minden érték külön "sorban" van, és minden sor egyetlen oszlopot tartalmaz. Az első dimenzió a minták száma (batch size), amásodik dimenzió a jellemzők száma (features).
- Egyszerű lineáris modell: egyetlen Dense réteggel, melynek 1 bemeneti és 1 kimeneti neuronja van (y = wx + b formában, ahol w a súly és b az eltolás).

```
# SGD optimizáló algoritmus, loss és metrics meghatározása
model.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=0.01), loss='mean_squared_error',
metrics=['mean_absolute_error'] )
# Tanítás
model.fit(X, y, epochs=100, verbose=0)          # 100 epochon keresztül tanítjuk a modellt

# Modellparaméterek
weights, bias = model.layers[0].get_weights()          # a tanult súly és az eltolás lekérdezése
print(f"Learned weight: {weights[0][0]:.2f}, bias: {bias[0]:.2f}")

print("Predictions:")
predictions = model.predict(X)                    # a modell előrejelzése az X adatokra
print(predictions[:5])                    # az első 5 előrejelzés kiírása
print(y[:5])                    # az első 5 célérték kiírása
loss, mae = model.evaluate(X, y, verbose=0))     # a veszteség és a négyzetes eltérés átlaga a train adatokon
print(f"Loss (MSE): {loss:.4f}, MAE: {mae:.4f}")
```

- **SGD** (Stochastic Gradient Descent) az optimalizáló algoritmus.
- **MSE** (Mean Squared Error) a veszteségfüggvény, amelyet minimalizálni szeretnénk: **mean_squared_error** a négyzetes eltérések átlaga, ami nem százalékos arány, hanem egy numerikus hiba. Az alacsonyabb érték jobb illeszkedést jelent.
- **MAE** (Mean Absoluted Error): átlagos abszolút eltérés a predikciók és a valós értékek között. Egyszerű, skálafüggetlen hiba-mérték; a kisebb érték a jobb.
- A verbose=0 azt jelenti, hogy nem írunk ki részletes információt a tanítás során.
- A futási eredményeket lásd a következő dián.

```
# Vizualizáció
import matplotlib.pyplot as plt

plt.scatter(X, y, label='Data’)      # az eredeti adatok pontokként való megjelenítése
plt.plot(X, predictions, color='red', label='Model')  # a modell előrejelzéseinek megjelenítése vonalként
plt.title('Simple Linear Regression with TensorFlow')

plt.xlabel('X')
plt.ylabel('y')
plt.legend()
plt.show()
```

```
Learned weight: 2.07, bias: 2.78
Predictions:
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step
[[2.7793083]
 [2.9885483]
 [3.1977882]
 [3.4070282]
 [3.6162682]]
tf.Tensor(
[[3.5751219]
 [3.7890363]
 [3.792778 ]
 [3.6535974]
 [3.6916327]], shape=(5, 1), dtype=float32)
Loss (MSE): 0.2692, MAE: 0.4211
```



A modell tanítása:

200 epochon keresztül:

```
Learned weight: 1.99, bias: 2.84
Predictions:
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step
[[2.83688  ]
 [3.0378075]
 [3.238735 ]
 [3.4396625]
 [3.6405897]]
tf.Tensor(
[[2.596357 ]
 [3.062817 ]
 [3.6679618]
 [3.9370875]
 [2.8996828]], shape=(5, 1), dtype=float32)
Loss (MSE): 0.2990, MAE: 0.4454
```

500 epochon keresztül:

```
Learned weight: 2.00, bias: 2.96
Predictions:
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step
[[2.9600847]
 [3.1616607]
 [3.363237 ]
 [3.5648131]
 [3.7663891]]
tf.Tensor(
[[2.6862187]
 [3.1049225]
 [2.9444098]
 [3.933687 ]
 [3.8583324]], shape=(5, 1), dtype=float32)
Loss (MSE): 0.2229, MAE: 0.3757
```

4. példa: Két osztály szétválasztása MLP-vel (Multi-Layer Perceptron)

Cél: A modell értékelése és az MLP-vel két csoportba sorolt adatpontok vizualizálása.

```
import tensorflow as tf
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# Adatkészlet generálása (2 osztály)
X, y = make_classification(n_samples=500, n_features=2, n_classes=2, n_informative=2, n_redundant=0,
n_repeated=0, random_state=42)
X = tf.convert_to_tensor(X, dtype=tf.float32)
y = tf.convert_to_tensor(y, dtype=tf.float32)

# Adatok szétválasztása tanító és teszt halmazra
X_train, X_test, y_train, y_test = train_test_split(X.numpy(), y.numpy(), test_size=0.2, random_state=42)

# Visszaalakítás TensorFlow tensorokká
X_train = tf.convert_to_tensor(X_train, dtype=tf.float32)
X_test = tf.convert_to_tensor(X_test, dtype=tf.float32)
y_train = tf.convert_to_tensor(y_train, dtype=tf.float32)
y_test = tf.convert_to_tensor(y_test, dtype=tf.float32)
```

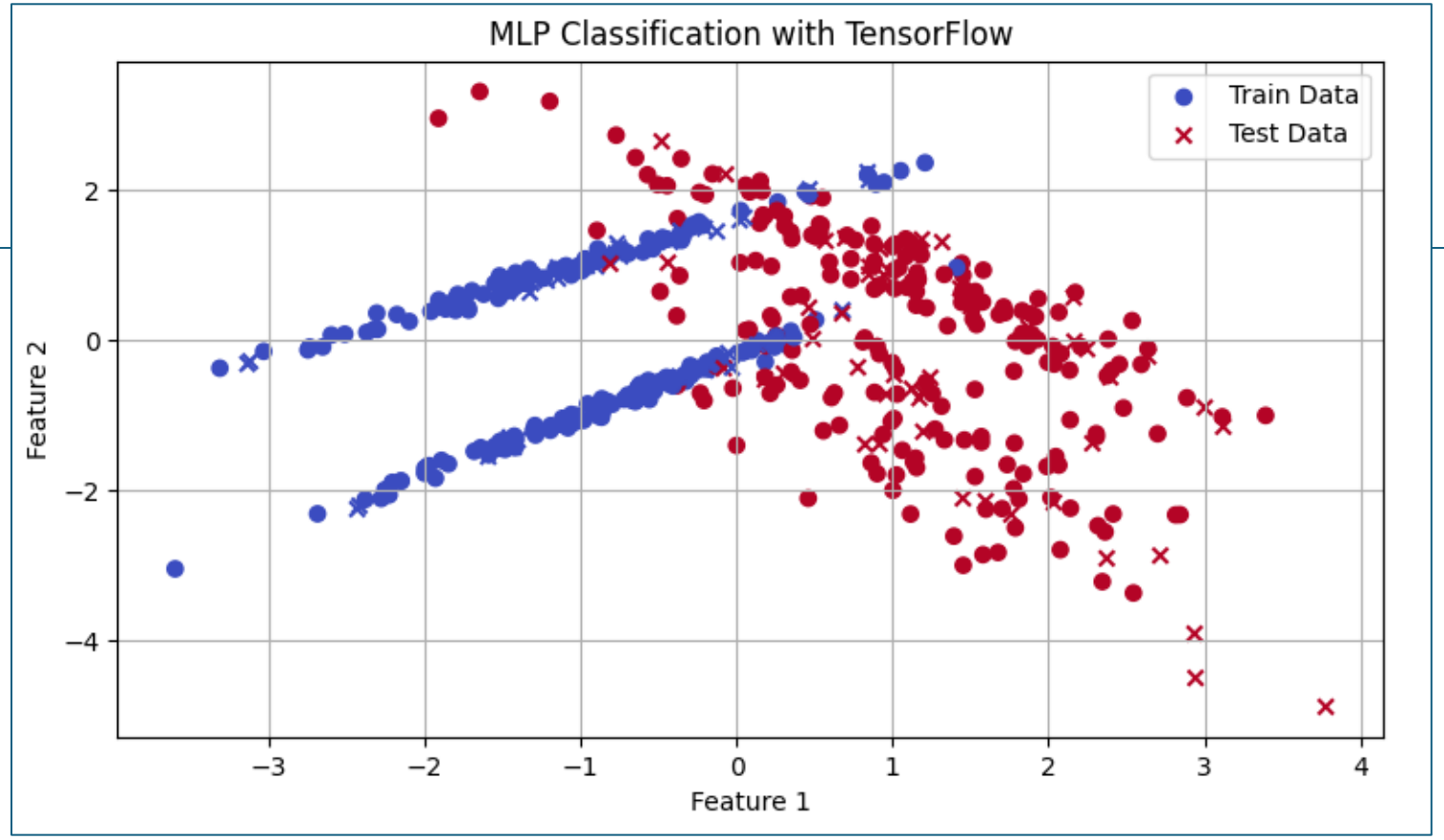
```
# Modell létrehozása
model = tf.keras.Sequential([
   tf.keras.layers.Input(shape=(2,)),                    # Bemeneti forma megadása külön Input rétegként
   tf.keras.layers.Dense(10, activation='relu'),            # 2 bemeneti jellemző
   tf.keras.layers.Dense(1, activation='sigmoid’)             # Kimenet sigmoid aktivációval
])
# Optimizáló, loss és metrics beállítása
model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
          loss='binary_crossentropy',
          metrics=['accuracy'])

# Tanítás
history = model.fit(X_train, y_train, epochs=100, batch_size=32, verbose=0, validation_data=(X_test, y_test))

# Modell értékelése
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"Final Loss: {loss:.4f}, Final Accuracy: {accuracy:.2f}")
```

```
Final Loss: 0.2075, Final Accuracy: 0.89
29931/29931 ━━━━━━━━━━━━━━━━━━━━ 13s 423us/step
```

Magyarázat:

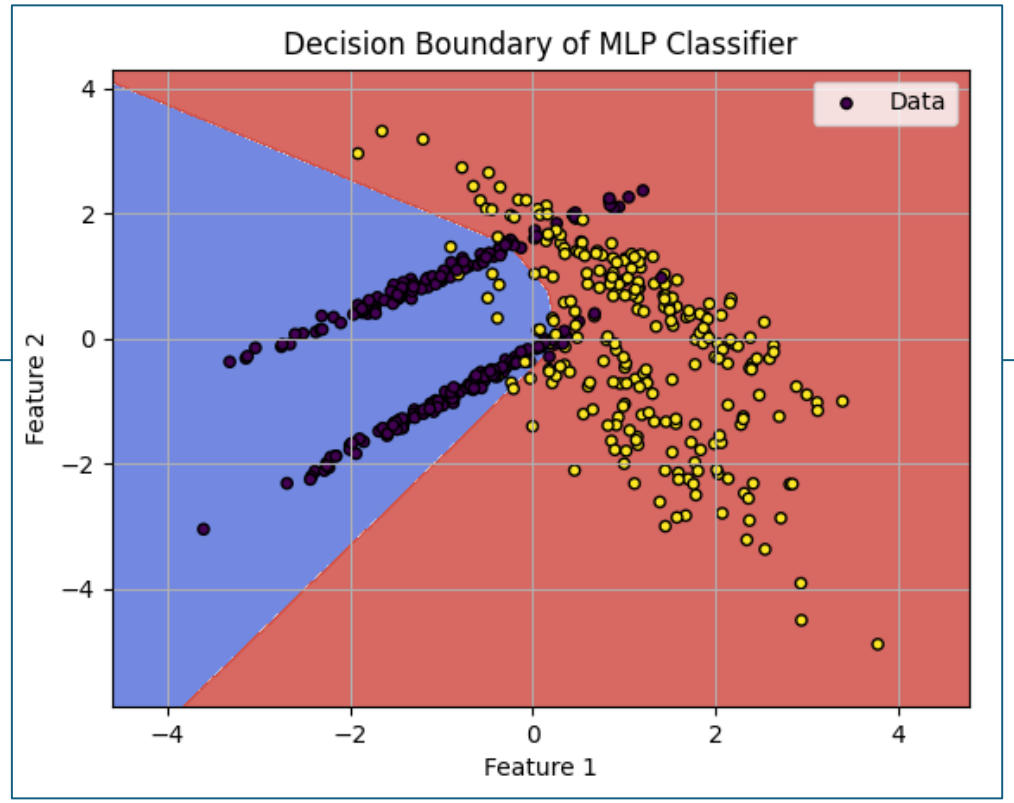
1. Adatkészlet generálása: A make_classification segítségével egy szintetikus bináris osztályozási adatkészletet hozunk létre. A TensorFlow tf.convert_to_tensor metódusával alakítjuk át az adatokat TensorFlow tensorokká.
2. Adatok szétválasztása: A train_test_split segítségével az adatokat tanító és teszt halmazra osztjuk.
3. Modell létrehozása: A tf.keras.Sequential egy egyszerű, rétegekből álló modellt hoz létre. Az első réteg egy 10 neuronos, **ReLU aktivációs függvényt** használó sűrű réteg. A második réteg egyetlen neuront tartalmaz sigmoid aktivációval, amely bináris osztályozáshoz szükséges.
4. Loss és optimizáló: A binary_crossentropy veszteségfüggvényt használjuk, amely bináris osztályozási problémákhoz megfelelő. Az **Adam optimalizáló** gyors és hatékony tanulást biztosít.
5. Tanítás: A model.fit metódus 100 epochon keresztül tanítja a modellt, 32-es batch mérettel. A validációs adathalmazt is megadjuk a tanítás során.
6. Értékelés: A model.evaluate segítségével kiértékeljük a modell teljesítményét a teszt adathalmazon.

```
# Adatok megjelenítése
plt.figure(figsize=(10, 6))
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap='coolwarm', label='Train Data')
plt.scatter(X_test[:, 0], X_test[:, 1], c=y_test, cmap='coolwarm', marker='x', label='Test Data')
plt.title('MLP Classification with TensorFlow')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.grid(True)
plt.show()
```



```
# Döntési határvonal megjelenítése
x_min, x_max = X.numpy()[:, 0].min() - 1, X.numpy()[:, 0].max() + 1
y_min, y_max = X.numpy()[:, 1].min() - 1, X.numpy()[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01), np.arange(y_min, y_max, 0.01))
Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = (Z > 0.5).astype(int)      # Bináris osztályozás
Z = Z.reshape(xx.shape)
plt.contourf(xx, yy, Z, alpha=0.8, cmap='coolwarm')
plt.scatter(X.numpy()[:, 0], X.numpy()[:, 1], c=y.numpy(), edgecolors='k', marker='o', s=20, label='Data')
plt.title('Decision Boundary of MLP Classifier')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.grid(True)
plt.show()
```

```
Final Loss: 0.2075, Final Accuracy: 0.89
29931/29931 ━━━━━━━━━━━━━━━━━━━━ 13s 423us/step
16/16 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step
```

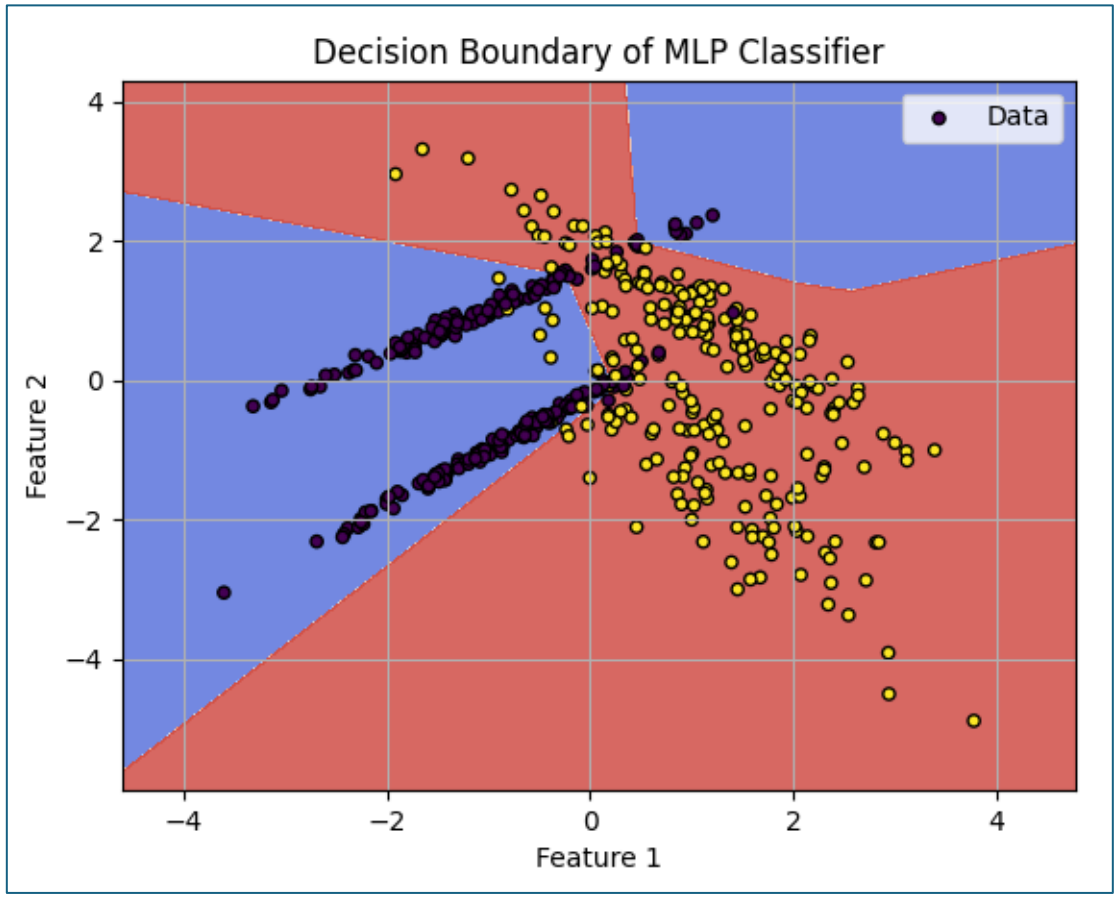


A döntési határvonal a két osztály közötti elválasztó vonal, amelyet a modell tanul meg az adatok alapján.

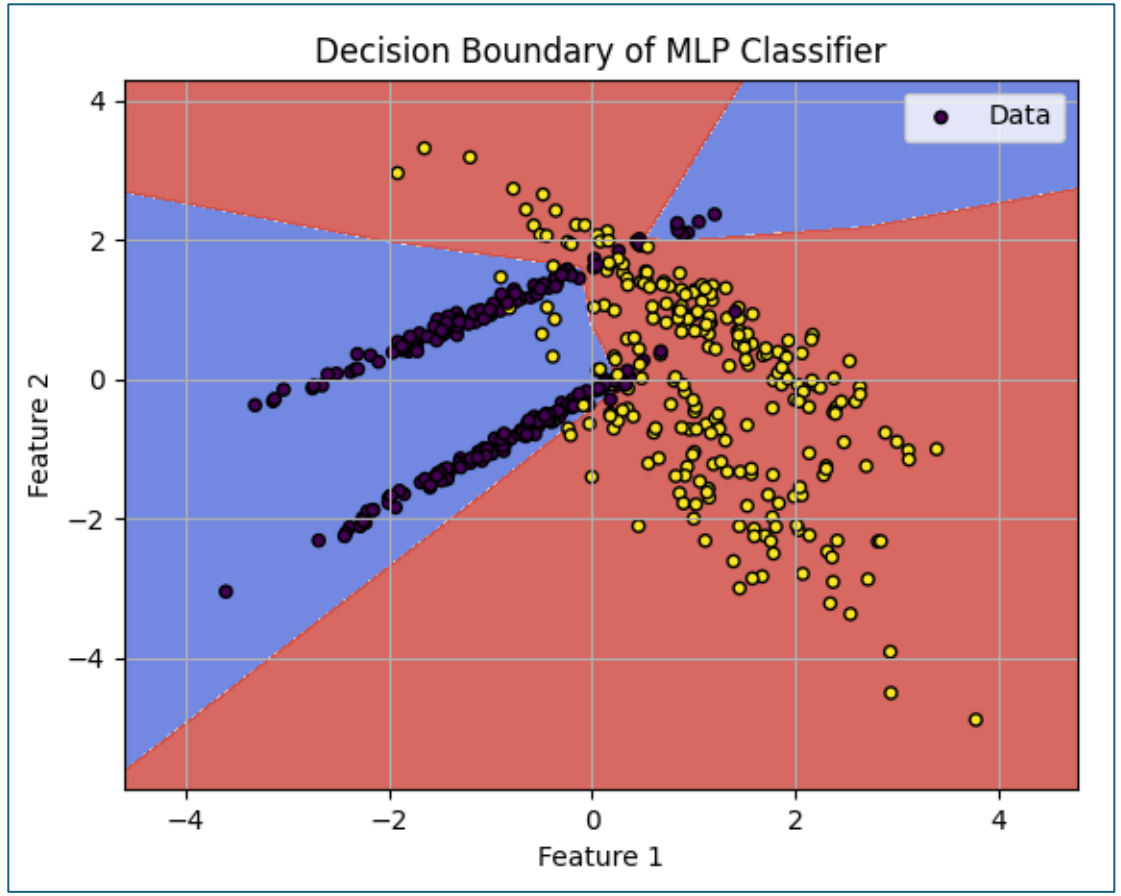
- A pontok színe a modell által előrejelzett osztályt jelzi.
- A piros és kék színek a két osztályt jelölik, köztük van a döntési határvonal.
- A modell a bemeneti adatok alapján tanulja meg a döntési határvonalat, amely elválasztja a két osztályt.
- A döntési határvonal a bemeneti jellemzők térében található, és a modell által tanult súlyok és eltolások alapján kerül meghatározásra.
- A döntési határvonal megjelenítése mutatja be, hogy a modell hogyan osztályozza az adatokat, és milyen jellemzők alapján hozza meg a döntéseit.

További futási eredmények:

Final Loss: 0.1888, Final Accuracy: 0.92



Final Loss: 0.1763, Final Accuracy: 0.93

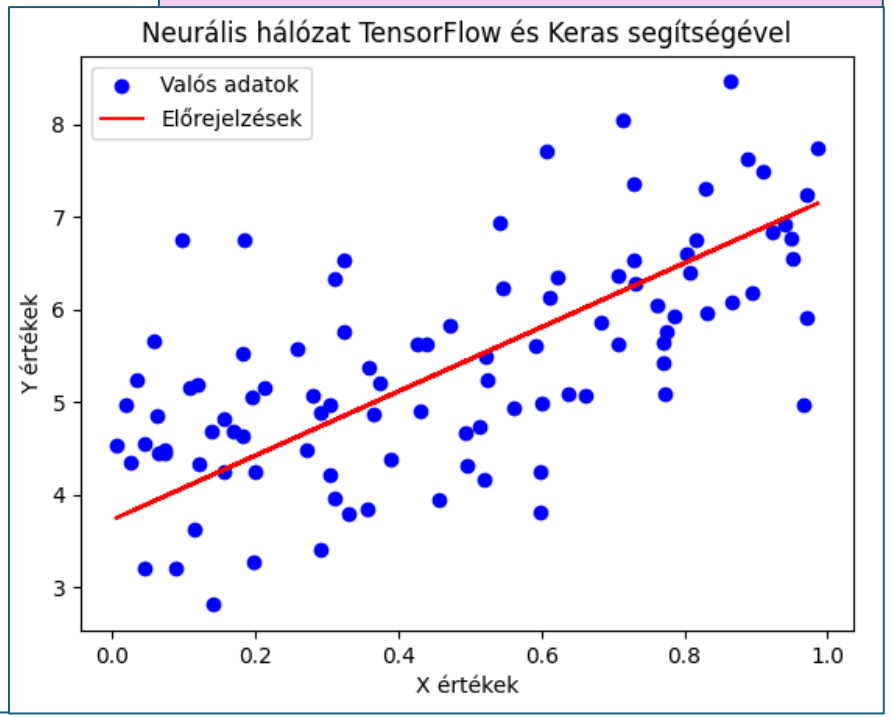


5. példa: egyszerű neurális hálózat létrehozása és betanítása

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Példa adatok létrehozása
np.random.seed(42)
x_np = np.random.rand(100, 1).astype(np.float32)
y_np = 4 + 3 * x_np + np.random.randn(100, 1).astype(np.float32)

# NumPy tömbök TensorFlow tensorokká alakítása
x_tf = tf.convert_to_tensor(x_np)
y_tf = tf.convert_to_tensor(y_np)

# TensorFlow modell létrehozása
model = Sequential([
   Dense(10, activation='relu', input_shape=(1,)),  # Bemeneti forma megadása külön Input
rétegként
   Dense(1)
])
```

```
# Modell fordítása
model.compile(optimizer='adam', loss='mean_squared_error')

# Modell betanítása
model.fit(x_tf, y_tf, epochs=500, verbose=0)

# Előrejelzések készítése
y_pred = model.predict(x_tf)
print('predikció',y_pred[:5])
print('valós értékek',y_np[:5])
print('Pontosság:', model.evaluate(x_tf, y_tf, verbose=0))
# a modell pontosságának kiírása a tanító adatokon

# Eredmények megjelenítése
import matplotlib.pyplot as plt
plt.scatter(x_np, y_np, color='blue', label='Valós adatok')
plt.plot(x_np, y_pred, color='red', label='Előrejelzések')
plt.xlabel('X értékek')
plt.ylabel('Y értékek')
plt.title('Neurális hálózat TensorFlow és Keras segítségével')
plt.legend()
plt.show()
```

```
predikció [[4.9417815]
 [7.1088247]
 [6.2861977]
 [5.784711 ]
 [4.1199026]]
valós értékek [[5.2106676]
 [6.5531354]
 [6.2877426]
 [3.8084068]
 [4.248384 ]]
Pontosság: 0.948832094669342
```



Köszönöm a figyelmet!

Konzultáció

Minden héten csütörtökön 18:00 – 20:00